Singularity 最佳实践

软件即是艺术

Michael.Pan

2025年12月15日

Singularity

- Singularity - 简介
- Singularity - 命令行安装
- Singularity - 常见命令使用
- Singularity - 打包 Python 命令行
- Singularity - 打包 Octave 程序
- Singularity - 打包 PI 并行计算程序

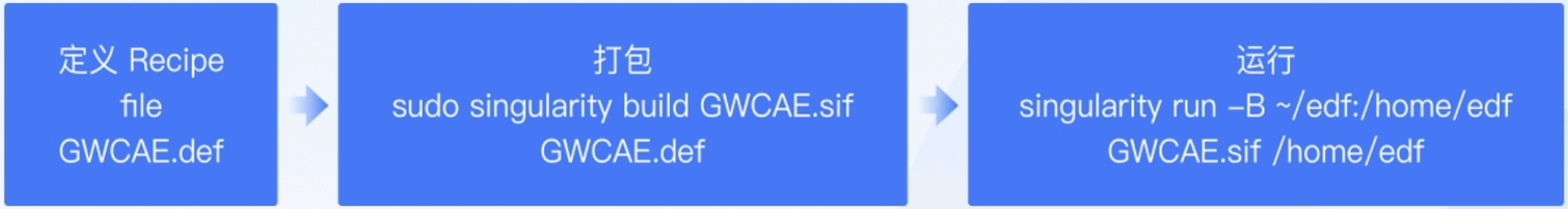
2

Singularity简介

Singularity是旨在以简单、可移植和可重现的方式在 HPC 集群上运行复杂的并行计算应用程序。Singularity目前是绝佳的 HPC 环境下的容器解决方案。它允许您创建和运行以可移植和可重现的方式打包软件的容器，然后在世界上许多最大的 HPC 集群、本地大学或公司集群、单个服务器、云中或大厅中的工作站上运行它。 你的容器是一个文件，你不必担心如何在每个不同的操作系统上安装你需要的所有软件

优势：

- 运行环境隔离
- 与宿主机系统无缝整合(权限，网络)
- 支持MPI并行
- 无需运行daemon进程
- 支持GUI软件，同时支持GPU加速



3

Singularity安装 - 1.1 两种安装方法

- 通过RPM or DEB 安装

SingularityCE 3.10.2 (https://github.com/sylabs/singularity/releases)

4

Singularity安装 - 1.2 配置修改

```
sudo sed -i 's/shared loop devices = no/shared loop devices = yes/g' \
/usr/local/etc/singularity/singularity.conf
```

5

Singularity命令 - 2.1 拉取预编译镜像

1）拉取ubuntu镜像

```
$ singularity pull docker://ubuntu:22.04
```

2）拉取centos镜像

```
$ singularity pull docker://centos:8
```

6

Singularity命令 - 2.2 运行镜像

交互式运行

```
$ singularity run ubuntu_22.04.sif
```

执行命令并退出

```
$ singularity run ubuntu_22.04.sif pwd
$ singularity run ubuntu_22.04.sif ls /bin
```

7

Singularity命令 - 2.3 挂载外部目录

```
$ singularity run -B /opt:/opt ubuntu_22.04.sif

体会一下不同
$ singularity run ubuntu_22.04.sif ls /opt
$ singularity run -B /opt:/opt ubuntu_22.04.sif ls /opt
```

8

Singularity打包命令行应用 - 3.1 目录

```
├── README.md
├── app
│   ├── hello.py
│   └── requirements.txt
├── hello.def
└── hello.sh
```

9

Singularity打包命令行应用 - 3.2 hello.py

```
1  import sys
2
3
4  def add(a, b):
5      return int(a)  + int(b)
6
7
8  if __name__ == "__main__":
9      m, n = sys.argv[1], sys.argv[2]
10      k = add(m, n)
11      print(k)
```

10

Singularity打包命令行应用 - 3.3 def文件

```
# hello.def
BootStrap: docker
From: harbor.michaelapp.com/library/ubuntu:22.04

%labels
    Author  yskj
    Version hello_v0.1

%environment

 %runscript
     exec python3 /opt/app/hello.py "${@}"

 %files
     ./app/hello.py            /opt/app/
     ./app/requirements.txt    /opt/app/

 %post
```

11

Singularity打包命令行应用 - 3.3 def文件

```
18  %post
19      export LC_ALL="C.UTF-8"
20      export DEBIAN_FRONTEND=noninteractive
21
22      # update images source
23      sed -i 's/archive.ubuntu.com/mirrors.huaweicloud.com/g' /etc/apt/sources.list
24      sed -i 's/security.ubuntu.com/mirrors.huaweicloud.com/g' /etc/apt/sources.list
25      apt update
26
27      # install python3 and requirements.txt
28      cd /opt/app
29      apt install python3 python3-pip -y
30      pip install -r requirements.txt -i  https://mirrors.aliyun.com/pypi/simple/
31
32  %help
33      hello: singularity run hello_v0.1.sif 10 20
```

12

Singularity打包命令行应用 - 3.4 打包

```
$ cd ../tutorials/tutorial01/
$ sudo singularity build hello_v0.1.sif hello.def
```

13

Singularity打包命令行应用 - 3.5 运行

```
$ singularity run hello_v0.1.sif 10 20
30
$ singularity run hello_v0.1.sif 66 100
166
```

14

Singularity打包Octave（类似Matlab程序）计算程序 - 4.1 目录

```
.
├── README.md
├── app
│   ├── gdyadd.m
│   └── main.m
├── run.sh
├── sumq.def
└── sumq.sh
```

15

Singularity打包Octave（类似Matlab程序）文件 - 4.2 main.m

```
filename = 'input.dat';
fileID = fopen(filename);
arr = fscanf(fileID,'%f');
fclose(fileID);
c = gdyadd(arr(1), arr(2));
```

16

Singularity打包Octave（类似Matlab程序）文件 - 4.2 gdyadd.m

```
1  function result = gdyadd(n1, n2)
2  %此函数用于计算
3  %输入2个数字，输出两个数字的平方和
4
5  result  = n1*n1 + n2*n2
```

17

Singularity打包Octave（类似Matlab程序）运行 - 4.3 打包

```
sudo singularity build sumq_v0.1.sif sumq.def
```

18

Singularity打包Octave（类似Matlab程序）运行 - 4.4 运行

```
yypan@deepmind:tutorials/octave01$ singularity run sumq_v0.1.sif
result = 74
```

19

Singularity打包pi并行计算 - 5.1 目录

```
.
├── app
│   └── cpi.c
├── cpi.def
├── cpi.sh
├── cpi_v0.1.sif
├── README.md
└── run.sh
```

20

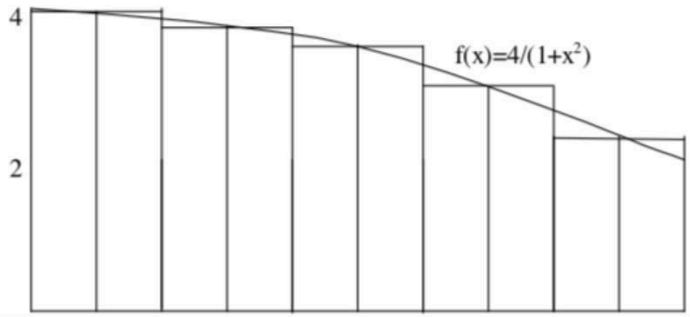
Singularity打包pi并行计算 - 5.2 算法

$$\int_0^1 \frac{1}{1+x^2} dx = \arctan(x)\,|_0^1 = \arctan(1) - \arctan(0) = \arctan(1) = \frac{\pi}{4}$$

令函数f(x)=4/(1+x²)，则有

$$\int_0^1 f(x)\,dx = \pi$$

而f(x)的图象为



计算f(x)图象下面从0到1之间的面积即为π的值。而该面积可以用图示的5个小矩形面积的和来近似，矩形的高度取函数在矩形中间点的取值，当用更多的矩形来划分时，该近似值就越接近于真实的π值。设将0到1的区间划分为N个矩形，则近似公式为：

$$\pi \approx \sum_{i=1}^{N} f\left(\frac{2\times i-1}{2\times N}\right) \times \frac{1}{N} = \frac{1}{N} \times \sum_{i=1}^{N} f\left(\frac{i-0.5}{N}\right)$$

https://blog.csdn.net/u013095333

Singularity打包pi并行计算 - 5.2 cpi.c

```
6  #include "mpi.h"
7  #include <stdio.h>
8  #include <math.h>
9
10  double f(double);
11
12  double f(double a)
13  {
14      return (4.0 / (1.0 + a * a));
15  }
16
17  int main(int argc, char *argv[])
18  {
19      int n, myid, numprocs, i;
20      double PI25DT = 3.14159265358979323846264643;
21      double mypi, pi, h, sum, x;
22      double startwtime = 0.0, endwtime;
23      int namelen;
24      char processor_name[MPI_MAX_PROCESSOR_NAME];
25
26      printf("cpi run start.\n");
27      MPI_Init(&argc, &argv);
```

Singularity打包pi并行计算 - 5.2 cpi.c

```
27      MPI_Init(&argc, &argv);
28      MPI_Comm_size(MPI_COMM_WORLD, &numprocs);
29      MPI_Comm_rank(MPI_COMM_WORLD, &myid);
30      MPI_Get_processor_name(processor_name, &namelen);
31
32      fprintf(stdout, "Process %d of %d is on %s\n", myid, numprocs, processor_name);
33      fflush(stdout);
34
35      n = 2000000000;  /* default # of rectangles */
36      if (myid == 0)
37          startwtime = MPI_Wtime();
38
39      MPI_Bcast(&n, 1, MPI_INT, 0, MPI_COMM_WORLD);
40
41      h = 1.0 / (double) n;
42      sum = 0.0;
43      /* A slightly better approach starts from large i and works back */
44      for (i = myid + 1; i <= n; i += numprocs) {
45          x = h * ((double) i - 0.5);
46          sum += f(x);
47      }
48      mypi = h * sum;
49
50      MPI_Reduce(&mypi, &pi, 1, MPI_DOUBLE, MPI_SUM, 0, MPI_COMM_WORLD);
```

Singularity打包pi并行计算 - 5.2 cpi.c

```
50      MPI_Reduce(&mypi, &pi, 1, MPI_DOUBLE, MPI_SUM, 0, MPI_COMM_WORLD);
51
52      if (myid == 0) {
53          endwtime = MPI_Wtime();
54          printf("pi is approximately %.16f, Error is %.16f\n", pi, fabs(pi - PI25DT));
55          printf("wall clock time = %f\n", endwtime - startwtime);
56          fflush(stdout);
57      }
58
59      MPI_Finalize();
60      printf("cpi run end.\n");
61      return 0;
```

Singularity打包pi并行计算 - 5.3 def文件

```
# cpi.def
BootStrap: docker
From: harbor.michaelapp.com/library/ubuntu:22.04

%labels
    Author  yskj
    Version cpi_v0.2

%environment
     export OMPI_DIR=/opt/ompi
     export PATH=$OMPI_DIR/bin:/opt/app:$PATH
     export LD_LIBRARY_PATH=$OMPI_DIR/lib:$LD_LIBRARY_PATH
     export OMPI_ALLOW_RUN_AS_ROOT=1
     export OMPI_ALLOW_RUN_AS_ROOT_CONFIRM=1

 %runscript
 SINGULARITYBIN=singularity
 ORTED_BIN=$(which orted)

 if [ "x$SINGULARITY_BIND" != "x" ];then
         SINGULARITY_BIND_OPT=" --bind $SINGULARITY_BIND"
 else
         SINGULARITY_BIND_OPT=""
 fi

 export OMPI_MCA_orte_launch_agent="$(cat <<EOF
 $SINGULARITYBIN exec $SINGULARITY_BIND_OPT $SINGULARITY_CONTAINER $ORTED_BIN
```

```
EOF
)"

exec $@

%files
```

25

Singularity打包pi并行计算 - 5.3 def文件

```
33  %files
34      ./app/cpi.c     /opt/app/
35      ./app/cpi.cxx   /opt/app/
36
37  %post
38      export LC_ALL="C.UTF-8"
39      export DEBIAN_FRONTEND=noninteractive
40      export OMPI_DIR=/opt/ompi
41      export PATH=$OMPI_DIR/bin:/opt/app:$PATH
42      export LD_LIBRARY_PATH=$OMPI_DIR/lib:$LD_LIBRARY_PATH
43      export OMPI_ALLOW_RUN_AS_ROOT=1
44      export OMPI_ALLOW_RUN_AS_ROOT_CONFIRM=1
45
46      # update images source
47      sed -i 's/archive.ubuntu.com/mirrors.huaweicloud.com/g' /etc/apt/sources.list
48      sed -i 's/security.ubuntu.com/mirrors.huaweicloud.com/g' /etc/apt/sources.list
49      # installing required packages
50      apt update && apt-get install -y wget git bash gcc gfortran g++ make file
51
52      # Installing Open MPI
53      mkdir -p /opt/ompi
```

Singularity打包pi并行计算 - 5.3 def文件

```
53      mkdir -p /opt/ompi
54      mkdir -p /tmp/ompi
55      cd /tmp/ompi
56      wget https://cae-static-1252829527.cos.ap-shanghai.myqcloud.com/devops/dev/openmpi-4.1.4.tar.gz
57      tar zxvf openmpi-4.1.4.tar.gz
58      cd openmpi-4.1.4/
59      ./configure --enable-mpi-cxx --enable-orterun-prefix-by-default --prefix=$OMPI_DIR && make -j8 &&
60      cd /opt/app/ && mpicc -o cpi cpi.c
61      cd /opt/app/ && mpicxx -o cpix cpi.cxx
62      rm -rf /tmp/ompi
63  %help
64      cpi: singularity run cpi_v0.2.sif /opt/app/cpi
```

27

Singularity打包pi并行计算 - 5.4 打包

```
$ cd ../tutorials/tutorial03/
$ sudo singularity build cpi_v0.1.sif cpi.def
```

28

Singularity打包pi并行计算 - 5.5 运行

```
$ singularity run cpi_v0.1.sif /opt/app/cpi
Process 0 of 1 is on cn01
pi is approximately 3.1415926535898389, Error is 0.0000000000000457
wall clock time = 7.680963

$ singularity run cpi_v0.1.sif mpirun -n 8 /opt/app/cpi
Process 4 of 8 is on cn01
...
Process 2 of 8 is on cn01
pi is approximately 3.1415926535897949, Error is 0.0000000000000018
wall clock time = 0.968542
```

29

Singularity打包pi并行计算 - 5.5 运行

```
$ singularity run cpi_v0.1.sif mpirun -n 16 /opt/app/cpi
Process 4 of 8 is on cn01
...
Process 13 of 16 is on cn01
pi is approximately 3.1415926535896892, Error is 0.0000000000001039
wall clock time = 0.486354

$ singularity run cpi_v0.1.sif mpirun -n 32 /opt/app/cpi
Process 4 of 8 is on cn01
...
Process 27 of 32 is on cn01
pi is approximately 3.1415926535898584, Error is 0.0000000000000653
wall clock time = 0.261127
```

30

Singularity打包pi并行计算 - 5.6 多节点运行

```
mpirun  --mca btl_tcp_if_include 192.168.0.0/24 \
--host master:48,cn01:16 singularity run cpi_v0.2.sif cpi

singularity run cpi_v0.2.sif mpirun --mca btl_tcp_if_include 192.168.0.0/24 \
--host master:48,cn01:32 cpi

Process 34 of 80 is on master
Process 44 of 80 is on master
...
Process 31 of 80 is on master
pi is approximately 3.1415926535897984, Error is 0.0000000000000053
wall clock time = 0.213129


备注：ip route show 查看通信路由
```

Singularity打包octave csv文件作为输入输出的算法

- https://github.com/panyingyun/octavesingularity

Singularity打包fortran csv文件作为输入输出的算法

- https://github.com/panyingyun/fortransingularity

Thank you

Michael.Pan
2025年12月15日